

An Algebraic Specification of **fsb**

Version 1.0

October 2, 2026

Describes the code at commit [2c4738a](#)

Abstract

This document specifies the security-relevant core of **fsb**, a local, read-only web view of a filesystem, as a small set of algebraic laws: the rule matcher that implements the two exclusion tiers (hide and deny), the access decision that every endpoint must agree with, the content-typed endpoints (images, PDFs, archives, Quick Look pictures), the pure frontend functions (ordering, filtering, navigation, link resolution), and the preview state machine. Each law is tied to an executable check. Deriving the laws, and then auditing every place that compares names, exposed sixteen defects, eight of them bypasses or gaps in the access rules, and a later independent review of the Quick Look endpoint two more; they are recorded in [Section 9](#).

Contents

1	Purpose and Method	2
1.1	Why an algebra	2
1.2	Method	2
1.3	What is not covered	3
2	Sorts and Notation	3
2.1	Sorts	3
2.2	Environment	3
2.3	Operations	3
2.4	Conventions	3
3	Names and Folding	4
3.1	The name equivalence	4
4	The Rule Algebra	4
4.1	Definitions	4
4.2	Laws for a rule set without negation (Deny)	5
4.3	Laws for every rule set	5
4.4	Boundaries	6
5	The Access Decision	6
5.1	One question, asked once	6
5.2	Laws	7
5.3	Boundaries	9

6	Content-Typed Endpoints	9
7	Frontend Functions	11
7.1	Row order	11
7.2	Filter	12
7.3	Navigation	12
7.4	Markdown links and images	12
8	The Preview State Machine	13
8.1	States and events	13
8.2	Transition table	13
8.3	Laws	14
9	Findings	15
9.1	Observations that are not defects	19
9.2	Defects found afterwards by review	19
9.3	What the method did and did not find	19
10	Refinement and Summary	20
10.1	From the model to the code	20
10.2	How to run the checks	20
10.3	Summary	20

1 Purpose and Method

1.1 Why an algebra

fsb’s central promise is negative: a path that the user has denied is never listed, searched, previewed, downloaded, or even confirmed to exist. That promise is a statement about *every* way of reaching a file, so it is threatened whenever two code paths answer the same question differently. The defects found and fixed in the project so far had exactly that shape: a listing through a symlinked directory that leaked denied names, a trailing newline that defeated a rule, an unreadable file answering 403 instead of 404, and a macOS firmlink alias that bypassed a rule.

An algebraic specification forces the question to be written down once, and then every operation to be compared with it. This document does that for four parts of the system:

1. the rule matcher (Section 4);
2. the access decision shared by all endpoints (Section 5);
3. the content-typed endpoints and the pure frontend functions (Sections 6 and 7); and
4. the preview state machine (Section 8).

1.2 Method

Each law is stated in the form *for all inputs of a stated kind, a stated equation or implication holds*, then given an executable check: a randomized property test with a fixed seed, a test against a fixture that assembles the awkward cases together, or an end-to-end browser test. Laws that failed

when first checked are recorded in Section 9 together with the fix. Laws that hold only under a stated assumption say so, and the assumptions that cannot be discharged inside the program are collected as *boundaries*.

1.3 What is not covered

The document does not model layout, streaming, timing, or the browser’s own behavior (except where the program relies on it). It does not replace an independent review; it is intended to give a reviewer a precise statement of what the program claims.

2 Sorts and Notation

2.1 Sorts

Name A path component: a finite sequence of Unicode characters containing neither / nor NUL.

Path An absolute path, written as the sequence of its components $p = \langle n_1, \dots, n_k \rangle$; the root is $\langle \rangle$. $\text{comps}(p)$ is that sequence, $\text{parent}(p) = \langle n_1, \dots, n_{k-1} \rangle$, and $p \cdot n$ appends a component.

Type $\{\text{dir}, \text{file}\}$, the type of the final component.

Rule A parsed pattern in the gitignore dialect, possibly negated and possibly restricted to directories.

RuleSet A finite sequence of rules. Two sets are used: **Deny** (hard block, no negation) and **Hide** (cosmetic, negation allowed).

Entry A directory entry as shown in a listing.

Row An entry as the frontend sorts it: name, type, size, and modification time.

2.2 Environment

Let **Roots** be the finite set of served roots, each already resolved to its real location. The filesystem is treated as a partial function from paths to files and directories with symbolic links; $\text{real}(p)$ is the canonical real location of p after all symbolic links and aliases (such as the macOS firmlink prefix `/System/Volumes/Data`) are resolved, defined when p exists. **canon** reduces aliases of the same location to one spelling.

2.3 Operations

The operations analyzed are, on the server:

Open, List, Search, and the endpoints built on them,

namely file download, head, metadata, image and PDF preview, and archive listing; and on the client the pure functions **cmp** and **sort** (row order), **filter**, **toHash** and **fromHash**, **goto**, and **resolveLink**.

2.4 Conventions

A relation $x \approx y$ between names means *the filesystem treats x and y as the same name*; it is defined in Section 3. A statement “**Open**(p) fails as **NotFound**” means that the operation fails with an error that the HTTP layer answers with an identical 404.

3 Names and Folding

3.1 The name equivalence

The rules are matched against names that the operating system will treat as identical. On a case-insensitive APFS volume two names denote the same file exactly when they are equal after Unicode normalization and *full* case folding. Full folding differs from simple folding: it expands some characters into several.

Definition 3.1 (Fold). $\text{fold}(s) = \text{nfc}(\text{cf}(\text{nfc}(s)))$, where cf is Unicode full case folding. For names, $x \approx y \iff \text{fold}(x) = \text{fold}(y)$. The extension to paths applies fold to the whole string, which is equivalent to folding component by component because folding never introduces or removes a separator.

Law 3.2 (Idempotence). $\text{fold}(\text{fold}(s)) = \text{fold}(s)$.

Observation 3.3 (Measured aliases). The set of single characters that APFS folds onto ASCII names was measured by creating every ASCII name of length one and two, and then probing every character in the Unicode range U+0080–U+2FFFF. On the tested system the aliases are exactly the following; each row names the ASCII spelling that the character opens.

character	opens
ß (sharp s), capital sharp s (U+1E9E)	ss
long s (U+017F)	s
Kelvin sign (U+212A)	k
ligatures U+FB00–U+FB06	ff, fi, fl, ffi, ffl, st

This is full case folding, not simple folding: under simple folding ß would remain a distinct character.

Law 3.4 (Fold agrees on the measured aliases). *For every pair in Observation 3.3, $\text{fold}(\text{character}) = \text{fold}(\text{ASCII spelling})$. The server's fold (Go, `x/text/cases`) and the client's fold (JavaScript, lower-, upper-, then lower-casing) both satisfy the law.*

Checked by: rules/fold_test.go::TestFoldMatchesTheAliasesAPFSTreatsAsEqual; web/src/lib/laws.test.ts (filter agreement)

Boundary 3.5 (Folding is a property of the volume). The alias set is a fact about the file system, not about fsb. A case-sensitive volume distinguishes more names and a future OS release could change the table. The specification therefore treats $\approx$ as the *coarsest* equivalence the program must respect: comparing after folding can only over-match, and over-matching a deny rule fails closed.

4 The Rule Algebra

4.1 Definitions

A rule r has a compiled test $\text{hit}(r, q, t)$: whether r selects the path q whose final component has type t . It is evaluated on folded text (Definition 3.1) with a pattern that was folded the same way.

Definition 4.1 (Exclusion). For a rule set $\mathcal{R}$ and a path p of type t , let $\text{anc}(p) = \{q \mid q \text{ is a prefix of } p\}$, where each proper prefix has type `dir` and p has type t . The last rule of $\mathcal{R}$ that hits q decides q : it excludes q unless it is negated. Then

$$\text{Match}(\mathcal{R}, p, t) = \exists q \in \text{anc}(p) : q \text{ is excluded by } \mathcal{R}.$$

Because an excluded ancestor already makes the whole path excluded, a later negation cannot re-include something inside it, as in git.

Definition 4.2 (Match below). $\text{MatchBelow}(\mathcal{R}, f, p, t)$ is Match restricted to the prefixes of p that are strictly longer than f . It is used to apply hide rules while listing a directory: entering a hidden directory on purpose shows its contents.

4.2 Laws for a rule set without negation (Deny)

Law 4.3 (Deny monotonicity). *If $\mathcal{R} \subseteq \mathcal{R}'$ then $\text{Match}(\mathcal{R}, p, t) \Rightarrow \text{Match}(\mathcal{R}', p, t)$. Adding a deny rule never makes anything accessible.*

Checked by: `rules/laws_test.go::TestLawDenyIsMonotoneAndOrderIndependent`

Law 4.4 (Order independence). *If $\mathcal{R}'$ is a permutation of $\mathcal{R}$, then $\text{Match}(\mathcal{R}, p, t) = \text{Match}(\mathcal{R}', p, t)$.*

Checked by: `rules/laws_test.go::TestLawDenyIsMonotoneAndOrderIndependent`

Law 4.5 (No negation in deny). *Parsing a deny file that contains a negated rule fails. Together with Laws 4.3 and 4.4 this gives the security reading of a deny file: it is a plain set of exclusions.*

Checked by: `rules/laws_test.go::TestLawDenyRefusesNegation`

4.3 Laws for every rule set

Law 4.6 (Ancestor closure). *If $\text{Match}(\mathcal{R}, p, \text{dir})$ then $\text{Match}(\mathcal{R}, p \cdot n, t)$ for every name n and type t . This holds also when $\mathcal{R}$ contains negated rules.*

Checked by: `rules/laws_test.go::TestLawAncestorClosure`

Law 4.7 (Fold invariance). *If $\text{fold}(p) = \text{fold}(p')$ then $\text{Match}(\mathcal{R}, p, t) = \text{Match}(\mathcal{R}, p', t)$: a path is excluded exactly when any spelling that the file system treats as the same name is. In particular no spelling of a denied name (upper case, decomposed accents, long s, Kelvin sign, sharp s, ligatures) escapes a rule.*

Checked by: `rules/laws_test.go::TestLawMatchDependsOnlyOnTheFoldedPath; guard/foldalias_test.go`

Law 4.8 (Narrowing). $\text{MatchBelow}(\mathcal{R}, f, p, t) \Rightarrow \text{Match}(\mathcal{R}, p, t)$. *Ignoring ancestors can only remove matches.*

Checked by: `rules/laws_test.go::TestLawMatchBelowNeverAddsMatches`

Law 4.9 (Single component). *If a pattern contains neither $/$ nor $**$, then $\text{hit}(r, q, t)$ depends only on the final component of q . Glob classes never match $/$: a negated class excludes it, and a $/$ inside a class is a pattern error (fail closed).*

Checked by: `rules/laws_test.go::TestLawSlashlessPatternsStayWithinOneComponent; TestSlashInsideAClassIsAParseError`

Law 4.10 (Credential folders match wherever they are). *The core rules for credential folders (`.ssh`, `.aws`, `.gnupg`, `.kube`, `.config/gh`, Keychains, and the browser profiles) and files (`.netrc`) are not anchored to the current home directory: a path that contains one of them at any depth is excluded. The same secrets exist in a backup or clone of the home folder, in another account's home, and under a wrong `$HOME`, where an anchored rule protects nothing.*

Checked by: `guard/differential_test.go::TestCredentialFoldersAreDeniedWhereverTheyAre`

Law 4.11 (A rule file works as written or fails loudly). *For every rule file, either parsing fails with a message, or each rule line means what a reader would take it to mean. In particular no accepted line is a rule that can never match: a leading byte order mark and any line ending are normalized, and an inline comment, `~user/`, a quoted pattern, an invisible character or a lookalike of the slash are refused.*

Checked by: `rules/parse_audit_test.go`

Law 4.12 (The quick test never changes an answer). *Matching first asks whether any rule could match, using one combined expression per kind of rule, and tests slash-free patterns against the last path component only. These are optimizations: for every rule set without negation and every path, the result is that of consulting the rules one by one against the whole path.*

Checked by: `rules/laws_test.go::TestLawFastPathEqualsTheRulesOneByOne`

Law 4.13 (Trailing whitespace is not significant). *For every rule r and path q , $\text{hit}(r, q \cdot \text{"n"}, t) = \text{hit}(r, q \cdot \text{"n "}, t)$ for names whose trailing characters are white space (space, tab, newline). `macOS` permits such names and they must not slip past a rule for `*.pem`.*

Checked by: `rules/rules_test.go; guard/review_test.go`

4.4 Boundaries

Boundary 4.14 (Hard links). Rules name paths, so a hard link to a denied file is invisible to them. The guard therefore judges a regular file with more than one name by its *identity* (device and inode): if any name of the file is denied, every name is refused, in `Open` and in listings alike (Law 5.5). The other names come from an index of all multi-name files under the roots and under the protected credential locations, built in the background because visiting every file takes seconds. A multi-name file is served only if the index accounts for all of its names now: it found as many distinct names (by folder identity and folded name) as the file has links, none is denied, and each is still, when the file is used, a real path to that file. Otherwise it is refused: while the index is not ready, for a name outside every root and protected location or in a folder that could not be read, after a walk stopped at its limit, and after the names changed. Within one listing or search a verdict is reused for at most one second, so a name or size shown may lag a rename by that long; `Open` never reuses it. Not detected: a denied name outside every root and protected location, whose file is refused rather than served.

Boundary 4.15 (Case classes in ranges). A positive class range such as `[+-0]` can include `/` by range even though the class text does not. No shipped rule uses such a range; a future tightening would expand ranges before compiling.

5 The Access Decision

5.1 One question, asked once

Every endpoint asks the same question of the guard: may this path be read? The answer is defined here and every operation is then compared with it.

Definition 5.1 (Access). Let $c = \text{canon}(\text{clean}(p))$ be the request path made absolute, cleaned, and reduced to one alias spelling, and let $\rho = \text{real}(c)$ be the real location of what is actually opened.

The type t is the type of the opened object. Then

$$\text{Ok}(p) \iff \text{inRoots}(\rho) \wedge \neg \text{Match}(\text{Deny}, \rho, t) \wedge \neg \text{Match}(\text{Deny}, c, t) \wedge \text{Elig}(o),$$

where o is the opened object and $\text{Elig}(o)$ is the hard-link condition: it holds unless o is a regular file with $n > 1$ names, in which case it holds iff the index accounts for all of them *now*. That means that the index is ready, that it found n distinct names (by folder identity and folded name), that no deny rule matches the real path of any of them, and that each is still a real path to o (same device and inode, no symbolic link along it). If the index is not ready, or any part cannot be established, $\text{Elig}(o)$ is false: unknown means refused. Identity is (device, inode); no content is read.

Without Elig the formula would be a path-and-root predicate only. An allowed `ordinary.txt` that has a second name `.env` satisfies the first three conjuncts and is nevertheless refused, which is the point of the fourth (Law 5.5).

The check is made on the *real* location (so a symbolic link cannot lead into a denied place), and also on the *spelled* location (so a link *named* like a denied file is refused even though its target is fine). It is therefore conservative: a path is refused if either name is denied.

Definition 5.2 (Open, list, search). $\text{Open}(p)$ returns the opened object, or `NotFound`, exactly when $\text{Ok}(p)$ fails or the path does not exist, and otherwise a permission or type error that mentions nothing about denied paths. $\text{List}(d)$ is defined when $\text{Open}(d)$ is; it contains an entry e of d exactly when $\text{Visible}(d, e)$:

$$\begin{aligned} \text{Visible}(d, e) \iff & \neg \text{Match}(\text{Deny}, \rho_d \cdot e, t) \wedge \neg \text{MatchBelow}(\text{Hide}, \rho_d, \rho_d \cdot e, t) \\ & \wedge (e \text{ a symbolic link} \Rightarrow \text{Ok-target}(e)) \wedge \text{Elig}_\tau(e), \end{aligned}$$

where Elig_τ is Elig with one difference: within a single listing or search a verdict about a file may be reused for at most $\tau = 1$ second, after which the names are checked again. A listing is therefore a snapshot whose hard-link verdicts can be up to τ old; Open always evaluates Elig afresh, and $\rho_d = \text{real}(d)$: children are judged by where the directory really is, not by the link it was reached through. $\text{Search}(d, q)$ walks breadth-first from d , entering only visible unlinked directories whose real location is still the name they were queued under, and reports the visible entries whose folded name contains $\text{fold}(q)$. It also reports whether it was *complete*: not if a folder that was allowed could not be opened or read to the end, or was replaced by a link before it was entered; denied, vanished and cloud-only folders do not count.

5.2 Laws

Law 5.3 (Denied is absent). *If $\neg \text{Ok}(p)$ because of a deny rule, then p is not opened, not listed by $\text{List}(\text{parent}(p))$, and not reported by any Search .*

Checked by: guard/laws_test.go::TestLawDeniedIsAbsentEverywhere; guard/fuzz_test.go (independent inode and marker oracle)

Law 5.4 (Every spelling the OS resolves is refused). *For every secret s and every spelling p such that the operating system resolves p to the same object as s (case, folding aliases, decomposed accents, doubled separators, dot segments, trailing separators, the data-volume and symbolic-link aliases, named forks), every operation refuses p . The premise is decided by the OS (same inode), not by *fsb*, so the law does not depend on *fsb*'s own idea of a name. Dually, every such spelling of an ordinary file inside the root is served.*

Checked by: guard/differential_test.go (both directions; fails 1,536 times if the folding fix is reverted)

Law 5.5 (Every name of a denied file is denied). *If a regular file has several names and one of them is matched by a deny rule, then Open refuses the file under every name, and no listing shows it. If the names cannot be determined, or the index does not account for every one of them (as many distinct names as the file has links, each still a real path to it), the file is refused. Identity is the pair (device, inode), never a hash of the contents, so an ordinary copy of a secret is not treated as the secret and no file is read to decide.*

Checked by: guard/hardlink_test.go (all four rule kinds, links made after the index was built, links to files outside the roots, refusal while the index builds, bounded walk, names renamed or moved, names outside every location, the retry walk and cooldown, the one-second verdict reuse in listings and searches); e2e "a hard link to a denied file is refused under any name"

Law 5.6 (Indistinguishable from missing). *For every endpoint f , a denied path p and a missing path q receive the same status and body: $f(p) = f(q)$. This includes unreadable files inside a denied folder, which must not answer 403.*

Checked by: guard/laws_test.go::TestLawDeniedIsAbsentEverywhere; server/m2_test.go; e2e "answers 404, with no content, for denied paths on every endpoint"

Law 5.7 (Listed implies openable). *If e is listed in $\text{List}(d)$ and is not a dangling link, then $\text{Open}(d \cdot e)$ succeeds, provided the file system does not change between the two operations and the operating system's own access and type conditions hold (permissions, a regular file or folder). This is a snapshot property: within τ of a change, a listing may still show a multi-name file that a fresh Open refuses (the Hard links boundary). The converse is false by design: hidden entries are not listed but can be opened.*

Checked by: guard/laws_test.go::TestLawListedImpliesOpenable

Law 5.8 (Found implies listed). *If Search reports m , then m is an entry of $\text{List}(\text{parent}(m))$, for a file system that does not change between the two observations (and, for multi-name files, to within τ). Search therefore cannot show what browsing would not, because both go through one function.*

Checked by: guard/laws_test.go::TestLawSearchResultsAreListed

Law 5.9 (Hidden is reachable). *If $\text{Match}(\text{Hide}, p, t)$ but $\text{Ok}(p)$ holds, then $\text{Open}(p)$ succeeds. Hide rules are cosmetic and never restrict direct access.*

Checked by: guard/guard_test.go (hidden entries); e2e "lists the home folder without denied or ignored entries"

Law 5.10 (Containment). $\text{Ok}(p) \Rightarrow \exists r \in \text{Roots} : \text{real}(p) \text{ is } r \text{ or lies beneath } r$, where “is r or lies beneath r ” is the identity relation inRoots of Law 5.11, not a comparison of spellings. (Folding belongs to the name and rule model, where over-matching is safe; it is not used for this allow decision.)

Checked by: guard/guard_test.go; guard/firmlink_darwin_test.go

Law 5.11 (Containment is by identity). $\text{inRoots}(\rho)$ holds iff the leading components of ρ that correspond to a root name the same directory as that root (same device and inode). Comparing spellings would be right on a case-insensitive volume, but on a case-sensitive one it would admit a

sibling folder that differs from the root only in case. This is an allow decision, so unlike the deny rules it must not over-match.

Checked by: guard/casesens_test.go (needs FSB_CASE_SENSITIVE_VOLUME); guard/guard_test.go::TestInRoots

Law 5.12 (Alias reduction is by name equality). *canon recognizes the data-volume prefix and the firmlink directories component by component, comparing components with fold, so no spelling of /System/Volumes/Data/Users escapes reduction. Slicing by byte length does not have this property, because a folded spelling can be longer or shorter.*

Checked by: guard/foldalias_test.go::TestRootOfSlashHonoursDenyRulesThroughFoldedFirmlinkSpellings; guard/firmlink_darwin_test.go

Law 5.13 (The session is scoped to a random path). *Browsers scope a cookie by host and never by port, so a cookie for 127.0.0.1 would accompany every request to every other web server on that address. All URLs of fsb therefore live under a random 128-bit prefix / π / fixed at launch; the cookie is set with Path= π /; and a request whose path is not under the prefix is refused exactly like an unauthenticated one, whatever cookie it carries. A cookie that is sent only for paths under / π / is never sent to another server, which has no such path. The frontend forms every request relative to the page's own directory.*

Checked by: httpguard/httpguard_test.go::TestOnlyURLsUnderThePrefixExist, TestSessionCookieIsScopedToThePrefix; e2e "the session cookie is scoped to fsb's own prefix" (fails if the path is widened); web/src/lib/laws.test.ts::apiBase

Law 5.14 (Canonical form is stable). $\text{canon}(\text{canon}(p)) = \text{canon}(p)$.

Checked by: guard/laws_test.go::TestLawCanonPathIsIdempotent

Law 5.15 (Bounded work). *Search examines at most V directory entries (excluded ones included) and reports at most M results, and reports truncation when a bound stopped it; it never follows symbolic links to directories, so it terminates on cyclic links.*

Checked by: guard/search_test.go; e2e (100,000-file directory, optional)

5.3 Boundaries

Boundary 5.16 (Time of check). The file is opened first and its real path is then resolved and confirmed to name the same object (**SameFile**); the decision is taken on the object that is actually served. A rename between opening and resolving is treated as a race and refused.

Boundary 5.17 (Response timing). A denied path can be answered slightly faster than a missing one. Status and body are identical; timing is not equalized.

Boundary 5.18 (Other processes). Anything that runs as the user can read the same files. fsb is not a sandbox against local malware.

6 Content-Typed Endpoints

Three endpoints look inside a file: inline image preview, inline PDF, and archive listing. Each is defined by a predicate on the file's *bytes*, never its name, and each factors through **Open**, so

Laws 5.3 and 5.6 apply to them unchanged. A fourth, the Quick Look picture, hands bytes to another program; Law 6.7 states what it may be given.

Definition 6.1 (Sniff). $\text{sniff}_{\text{img}}$ accepts a byte sequence that begins with the PNG, JPEG, GIF, or WebP signature; $\text{sniff}_{\text{pdf}}$ accepts one that begins with %PDF-; $\text{sniff}_{\text{arc}}$ accepts the start of a zip, a tar (ustar at offset 257), or a gzip stream that contains a tar.

Law 6.2 (Content decides). *The endpoint succeeds only if the corresponding sniff accepts the file's leading bytes; the file name is never consulted. A file named `a.png`, `a.pdf` or `a.zip` with other contents is refused with 415, and a file with a different name but the right contents is accepted. SVG and HTML are never accepted.*

Checked by: `guard/m2_test.go (OpenImage)`; `guard/pdf_test.go`; `guard/archive_test.go`; `server/pdf_test.go`

Law 6.3 (Cloud-only files are never read). *For a file whose contents are stored only in the cloud, no endpoint reads it implicitly: the guard reports it as not downloaded before any read.*

Checked by: `guard/dataless tests`; `e2e (hover peek)`

Law 6.4 (Archive listing discloses names only). *The archive listing contains, for each member, its name, size, type and date, and never any member data; names are displayed as text with control characters, Unicode formatting characters and invalid UTF-8 replaced, are cut to a bounded prefix (4,096 bytes, then an ellipsis), and are never used as paths.*

Checked by: `guard/archive_test.go::TestListArchiveNamesAreDefanged`; `server/archive_test.go`

Law 6.5 (Archive listing is bounded). *For any file, listing examines at most E tar entries or zip directory records, streams at most D decompressed bytes of a compressed tar, reads at most B bytes of a zip directory, returns at most S entries, and reports (as incomplete) when a scan limit stopped it. A valid archive that exceeds a scan limit is listed up to the limit and marked incomplete; it is not refused. A zip whose directory was read in full is refused if its record count disagrees with the archive's end record, which can be checked only then. The byte bound B is checked between records, so the last record may cross it by at most one record's length, and the work stays bounded. A plain tar is read by skipping each member's body, so its cost does not grow with member sizes.*

Checked by: `guard/archive_test.go::TestListArchiveIsBounded`,
`TestListArchiveStopsAtTheDecompressionLimit`, `TestZipEntryLimitHoldsForValidArchives`,
`TestZipDirectoryByteBudget`, `TestZipWithAMisleadingCountIsRefused`,
`TestTarGzLimitAtAHeaderBoundaryIsIncomplete`,
`TestPlainTarListingStillSeeksPastMemberBodies`

Law 6.6 (Inline PDF is framed only by the app). *The PDF response carries `X-Frame-Options: SAMEORIGIN` and `frame-ancestors 'self'`, the app shell carries `DENY` and `frame-ancestors 'none'`, and the Markdown frame document carries its own policy that admits exactly its one script and one style by hash.*

Checked by: `server/pdf_test.go`; `server/mdframe_test.go`; `web/mdframe_test.go`; `e2e PDF and Markdown tests`

Law 6.7 (Quick Look sees only what Open serves). *Let $C(p)$ be what Quick Look is given for a request on p . $C(p)$ is defined only if `Open(p)` succeeds, the real path has an `iWork` or `Office` extension, and the file is not cloud-only. For a file, $C(p)$ is a new file in a private folder whose bytes are those read from the descriptor `Open(p)` returned. For a package, $C(p)$ is the set of pairs (r, b) such that*

Open(p/r) succeeds on a regular file whose real path lies under the real path of p, and b is the bytes read from that descriptor. In both cases C(p) carries no extended attributes and no resource fork, and it is never the path p itself. Hence every byte Quick Look reads is a byte some request could already read through Open, and Law 5.3 extends to it: no denied file and nothing outside the roots reaches it, whatever p names (an alias, a link, a package with hidden parts).

Checked by: server/quicklook_test.go (private copy, no extended attributes, package parts, a real Finder alias); server/fuzz_test.go::FuzzPathParameter (stand-in Quick Look echoes its input)

Boundary 6.8 (What Quick Look does with the copy). The law bounds Quick Look’s *input*, not its behavior: a flaw in Apple’s generators, or a document that refers to something outside itself, is not modeled. The independent review found no generator that loads local files named inside a document.

Boundary 6.9 (PDF sandboxing). A CSP `sandbox` cannot be applied to a PDF, because browsers then refuse to display it. The protection is instead the byte check, the framing headers, and the browser’s own isolated PDF viewer.

7 Frontend Functions

These functions are pure, so their laws are checked exhaustively enough by seeded random testing.

7.1 Row order

Let $\text{cmp}_{k,a,g}$ be the row comparison for sort key $k \in \{\text{name, size, time, kind}\}$, direction a , and grouping flag g (“Folders first”). A folder has no size and sorts as smaller than any file; a modification time that cannot be read sorts as the oldest; ties fall back to the name, then to the exact name, then to the raw characters (a collator that compares digits by value calls `file1` and `file01` equal, and the order must not depend on the input).

Law 7.1 (Total preorder). *For every k, a, g , cmp is antisymmetric ($\text{cmp}(x, y) = -\text{cmp}(y, x)$), transitive, and total. This holds even for rows whose time is unparseable.*

Checked by: web/src/lib/laws.test.ts: compareRows is antisymmetric and transitive

Law 7.2 (Input order does not matter). *For distinct names, $\text{sort}(\pi(xs)) = \text{sort}(xs)$ for every permutation π . (Two names that differ only by normalization cannot coexist in one directory on APFS, so the premise excludes them.)*

Checked by: web/src/lib/laws.test.ts: sortRows is independent of input order

Law 7.3 (Grouping). *If g , every folder precedes every file, for every key and both directions; grouping does not flip with the direction.*

Checked by: web/src/lib/laws.test.ts: Folders first groups folders

Law 7.4 (Reversal). *If $\neg g$, the descending order is the reverse of the ascending order.*

Checked by: web/src/lib/laws.test.ts: descending is the reverse of ascending

7.2 Filter

Law 7.5 (Filter agrees with search). *For case-insensitive matching, $\text{filter}(n, q) \iff \text{fold}(n) \ni \text{fold}(q)$, where the client's fold equals the server's on the aliases of Observation 3.3. The filter box therefore finds the same names that filename search finds. With “Match case” both sides compare after normalization only.*

Checked by: web/src/lib/laws.test.ts: the filter agrees with search; guard/matchcase_test.go

7.3 Navigation

Law 7.6 (Hash round trip). *$\text{fromHash}(\text{toHash}(p, s)) = (p, s)$ for every path and selection made of names, including names containing %, #, ?, @, =, quotes, spaces, and non-ASCII characters. A literal ? in a path is always encoded, so the first ? of a hash starts the options.*

Checked by: web/src/lib/laws.test.ts: parseHash inverts pathToHash; format.test.ts

Law 7.7 (Go to path is clean and idempotent). *If $\text{goto}(s) = p$ then p is absolute, contains no ., .. or empty component (except the root itself), never climbs above the root, and $\text{goto}(p) = p$.*

Checked by: web/src/lib/laws.test.ts: resolveGoto is clean and idempotent; goto.test.ts

Law 7.8 (Roots are prefix closed). *p is within a root r iff $p = r$ or p begins with r followed by /, so a sibling sharing a prefix (/Users/mex against /Users/me) is outside. The client check is advisory; the server's inRoots decides.*

Checked by: web/src/lib/laws.test.ts: withinRoots

Law 7.9 (A name is shown as what it is). *No character that reorders or hides text (controls, bidirectional overrides and isolates, line and paragraph separators, the byte order mark, tag characters) is sent to the screen inside a file name: each is replaced by a visible marker naming its code point. Text that needs none of them, including right-to-left scripts, is unchanged, and links and downloads use the real name.*

Checked by: web/src/lib/laws.test.ts: displayName; e2e "names that would disguise themselves are shown with visible markers"

7.4 Markdown links and images

Law 7.10 (Links resolve to normalized absolute paths). *If $\text{resolveLink}(b, h)$ names a file, the path is absolute and contains no ., .. or empty component; relative targets are resolved against the directory b of the Markdown file and cannot climb above the root.*

Checked by: web/src/lib/laws.test.ts: resolveLink yields normalized absolute file paths; markdown.test.ts

Law 7.11 (Links and images carry no address). *The generated markup has no href and no src on links or images before sanitizing; remote images are replaced by a placeholder, raw HTML is escaped, and only http and https links are ever opened, in a new tab without a referrer. Only base64 raster data: URLs fetched through the guarded preview endpoint are put into images.*

Checked by: web/src/lib/markdown.test.ts; e2e Markdown tests

8 The Preview State Machine

8.1 States and events

For a selected entry the preview pane is in one of the states

idle, loading, folder, image, quicklook, pdf, archive, text, binary, empty, dataless, error.

The name of the entry gives a *hint* $h \in \{\text{quicklook, image, pdf, archive, none}\}$; it is only a hint because the server decides (from the bytes, or for Quick Look from what Quick Look can draw). The state **loading** is written below as four sub-states, because what the pane is waiting for differs: **loading_{wait}**, the short delay after a selection that keeps holding an arrow key from firing a request per row (120 ms), and **loading_{pdf}**, **loading_{arc}** and **loading_{head}**, a request in flight. The pane displays all four alike, as “Loading”.

8.2 Transition table

Table 1 is the complete transition relation, read from `web/src/lib/Preview.svelte`. The code has no table of its own: the state is one variable, set where each answer arrives, so this table is the place where all the transitions can be seen together. Each row is a state, an event in it, the next state, and the request that the transition starts. The details request (`api/meta`) is separate: it starts with every transition out of **loading_{wait}**, never changes the state, and has its own error message.

Table 1: Transitions of the preview pane.

	State	Event	Next state	Request started
T1	any	the selection is cleared	idle	none
T2	any	an entry is selected	loading_{wait}	none; everything started for the previous selection is cancelled
T3	loading_{wait}	120 ms pass; hint quicklook and not a broken link	quicklook	<code>api/quicklook</code> (as a picture)
T4	loading_{wait}	120 ms pass; otherwise a folder	folder	none
T5	loading_{wait}	120 ms pass; otherwise a broken link	error	none
T6	loading_{wait}	120 ms pass; otherwise hint image	image	<code>api/preview</code> (as a picture)
T7	loading_{wait}	120 ms pass; otherwise hint pdf	loading_{pdf}	<code>api/pdf</code> , first byte only
T8	loading_{wait}	120 ms pass; otherwise hint archive	loading_{arc}	<code>api/archive</code>
T9	loading_{wait}	120 ms pass; otherwise	loading_{head}	<code>api/head</code>
T10	quicklook	the picture fails to load, and the entry is a folder (a package)	folder	none
T11	quicklook	the picture fails to load, and the entry is a file	loading_{head}	<code>api/head</code>
T12	image	the picture fails to load	loading_{head}	<code>api/head</code>
T13	loading_{pdf}	answer 200 or 206 of type <code>application/pdf</code>	pdf	<code>api/pdf</code> (in a frame)
T14	loading_{pdf}	answer 409	dataless	none

	State	Event	Next state	Request started
T15	loading _{pdf}	any other answer, or no answer	loading _{head}	api/head
T16	loading _{arc}	a listing	archive	none
T17	loading _{arc}	answer 409	dataless	none
T18	loading _{arc}	any other answer, or no answer	loading _{head}	api/head
T19	loading _{head}	an answer of kind $k \in \{\text{text, binary, empty, dataless}\}$	text, binary, empty or dataless, as k	none
T20	loading _{head}	any failure	error	none
T21	any	an answer or a picture failure for a cancelled selection	unchanged	none

No other transition exists. In particular, no row leaves `folder`, `pdf`, `archive`, `text`, `binary`, `empty`, `dataless` or `error` except T1 and T2 (a new selection), and no row enters `loadinghead` from `loadinghead`. `image` and `quicklook` are final unless their picture fails (T10–T12); a PDF the viewer cannot display stays in `pdf`, the viewer reporting the problem itself.

8.3 Laws

Law 8.1 (Termination). *From loading every run reaches a terminal state (folder, image, quicklook, pdf, archive, text, binary, empty, dataless, or error) after at most two requests: one hinted request and at most one fall back to the head request, which never falls back again. In Table 1: every path from loading_{wait} passes through at most one of T3, T6, T7 or T8 and at most one entry into loading_{head}, from which T19 and T20 lead only to final states.*

Law 8.2 (Freshness). *The content shown always belongs to the entry that is currently selected: every request started for a selection, including the fall back after an image or a Quick Look picture fails to load, is cancelled when the selection changes, and a cancelled request never changes the state. (T2 and T21 in Table 1.)*

Law 8.3 (Frame only after confirmation). *An inline frame is created for a PDF only in state pdf, which is entered only after the server has answered application/pdf for that path; a Markdown frame is created only for a text head whose name is a Markdown name, and receives only sanitized HTML.*

Checked by: e2e: "shows a real PDF in the built-in viewer frame, and treats a fake .pdf as text"; Markdown tests

Law 8.4 (Nothing is shown for a denied entry). *Every request of the state machine goes through the endpoints of Section 5; for a denied entry each answers as for a missing one (Law 5.6), so the pane shows only "Not available".*

Checked by: e2e: never lists secret files; answers 404

Boundary 8.5 (Termination and freshness are argued, not enumerated). Laws 8.1 and 8.2 are properties of the component’s control flow. Freshness was violated by one path (Finding 2) and is now guaranteed by keeping a single cancellation handle per selection; there is no automated test for that race, because it needs a controllable slow response. It is exercised indirectly by the end-to-end tests that select files quickly.

9 Findings

Stating the laws and then checking them found six defects (1–6), and a subsequent audit of every place that compares names or paths, and of the environment assumptions behind the rules, found ten more (7–16). A second independent review, of the Quick Look endpoint added later, found two more (17–18) against Laws 5.3 and 5.6. Each was reproduced by a failing test before it was fixed. Findings 1, 7, 8, 10 to 15, 17 and 18 are security defects; the rest are consistency defects that fail closed or are cosmetic, but each is a place where two parts of the program disagreed.

	Defect	Law broken	Fix
1	Deny bypass through full case folding. On APFS the name <code>.ssh</code> opens <code>.ssh</code> , but the matcher folded with simple folding, so a rule for <code>~/ssh</code> did not match it; <code>Open</code> , <code>Head</code> and the listing of the parent all served the denied folder. Also long s, the Kelvin sign and the ligatures.	4.7, 5.3	Rules, roots and search compare after full folding (<code>rules.Fold</code>); the alias set was measured on this file system and is a test.
2	Listed but unopenable. A plain file named like a <code>dir/</code> deny rule (a file called <code>cache</code>) was shown by <code>List</code> but refused by <code>Open</code> , because <code>Open</code> 's pre-check assumed every path is a directory.	5.7	The pre-check treats the final component as a file; the precise check after opening still refuses real directories.
3	Glob class matched across a slash. <code>[!a]</code> compiled to <code>[^a]</code> , which matches <code>/</code> , so a pattern with no slash could match across directory components.	4.9	Negated classes exclude <code>/</code> ; a <code>/</code> inside a class is a pattern error.
4	Row order not transitive. A modified time that could not be parsed compared equal to everything, so <code>a=b</code> , <code>a=c</code> but <code>b<c</code> . The resulting order depended on the input order.	7.1, 7.2	Unreadable times sort as the oldest; comparisons no longer use subtraction.

	Defect	Law broken	Fix
5	Filter and search disagreed. The filter box folded with simple lower-casing, so <code>strasse</code> did not find <code>Straße</code> , while filename search did.	7.5	The filter folds with lower-, upper-, lower-casing, which matches full folding on the measured aliases.
6	Stale fallback. After an image failed to load, the fallback request had its own cancellation handle, so a slow answer could replace the preview of a newer selection.	8.2	One handle per selection is shared by every request started for it.
7	Firmlink prefix matched by byte length. The data-volume prefix and the firm-link directories were recognized by slicing the path to the prefix's length, so a folded spelling (<code>/Slong-system</code> , or <code>Data</code> with the <code>st</code> ligature) was cut mid-character and not reduced. Under <code>-root /</code> the unreduced path was inside the root and matched no rule.	5.12, 5.3	Components are compared one at a time with <code>Fold</code> .
8	Root containment by spelling. Containment folded names, so on a case-sensitive volume a root <code>Public</code> also served the different folder <code>public</code> . Folding is safe for deny rules but not for an allow check.	5.11, 5.10	Containment is decided by identity: the corresponding leading directory must be the root itself.
9	Client and CLI comparisons. Breadcrumbs sliced the path by the root's character count (wrong when folding changes length) and compared roots case-sensitively; a link or hash naming a file with another case or accent selected nothing; the core-rule warning compared rule text exactly.	7.8	All use the same name equality; breadcrumbs split by components.

	Defect	Law broken	Fix
10	Credential rules anchored to one home directory. The rules for <code>~/.ssh</code> , <code>~/.aws</code> and the others named the current home only, so the same folders in a backup or clone of the home folder, in another account's home, or under a wrong <code>\$HOME</code> were served.	4.10	The credential rules match at any depth, like the secret-file patterns.
11	A path is not a file (hard links). Rules name paths, so a hard link to a credential file placed anywhere else was served under its innocent name; this had been recorded only as a limitation.	5.3, 5.5	A file with several names is judged by identity: a denied name denies them all, in <code>Open</code> and in listings. First done for the credential locations only.
12	The session cookie followed the host, not the port. Cookies are scoped by host, so the cookie was also sent to every other web server on <code>127.0.0.1</code> that the same browser visited, which could replay it. It had been recorded as a limitation.	5.13	Everything is served under a random per-launch prefix and the cookie's path is that prefix; requests outside it are refused.
13	Hard links to <code>.env</code>, <code>.pem</code> and <code>.key</code> files. The identity check of finding 11 covered only the credential folders, so a second name for a secret file elsewhere (<code>notes.txt</code> linked to a <code>.env</code>) was still served.	5.5	The check covers every denied name under the roots and protected locations, using a background index of multi-name files; multi-name files are refused until the index is ready.

	Defect	Law broken	Fix
14	Rule files that load and do nothing. A byte order mark broke the first rule; bare carriage returns joined lines; an inline comment, <code>~user/</code> , quotes, an invisible character or a lookalike slash produced a rule that never matched. All loaded without an error.	4.11	The byte order mark and line endings are normalized; the rest are errors that say what is wrong, so fsb does not start.
15	Secrets under other names. Only known folders and the extensions <code>.pem</code> and <code>.key</code> were denied, so a private key copied out of <code>.ssh(id_rsa)</code> , a certificate store or a password database was served.	4.10	The names that identify such files are core rules at any depth (<code>.pub</code> files stay reachable).
16	A name that shows as another name. A right-to-left override or a control character in a file name reordered or hid what the listing displayed, so <code>invoiceU+202Etxt.exe</code> read as a text file.	7.9	Deceptive characters are replaced, for display only, by a visible marker; real right-to-left text is untouched and downloads keep the real name.
17	A path that means more to another program. Quick Look was given the real path of a document that passed every check, but a Finder alias named <code>report.docx</code> is a regular file that Quick Look resolves to its target, so it drew a file in a denied folder, or outside the roots.	5.3	Quick Look is given only a private copy of bytes read through Open (Law 6.7); a real alias is the regression test.
18	A package answered differently for a hidden part. A package document holding a denied file was refused (404) while a clean one answered 200, although listings show both alike.	5.6	Parts a listing leaves out are left out of the copy, and the package answers as a clean one.

9.1 Observations that are not defects

- A dangling symbolic link is listed (marked as broken) but cannot be opened. Law 5.7 excludes it.
- Hidden entries are not listed but can be opened, and a hidden directory that is entered on purpose shows its contents. (Law 5.9.)
- A symbolic link *named* like a denied file is refused even when its target is harmless. Definition 5.1 checks both names.
- The macOS by-inode path `/.vol/<device>/<inode>` was probed as a possible alias that would evade path rules; on the tested system the directory is empty and the path cannot be opened.

9.2 Defects found afterwards by review

Reviews of the code (October 2026) found defects that these laws did not, each in a place the laws state only loosely: a hard-link verdict served for a file whose names the index had not seen, or whose names changed (Law 5.5 now says what “determined” means); a search that entered a folder swapped for a link after it was queued, and that looked complete when a folder could not be opened (the search definition now says so); a collator that left two names in input order (the row order now ends on the raw characters); and several limits that did not bind (a count that could never trip, a work limit that ignored excluded entries). The design specification’s history lists them; each has a test that fails without its fix.

9.3 What the method did and did not find

Findings 1 to 3 come from the rule and access laws and are about names: two spellings, two types, two segments treated as one. Findings 4 to 6 come from the frontend laws. Findings 7 to 9 came from asking of every comparison in the code whether it assumes that the same name means the same bytes, and from checking each answer against the real file system (including a case-sensitive volume). Findings 1, 7 and 8 share one lesson: name equality is a property of the volume, so deny checks may over-match but allow checks must decide by identity. Finding 17 adds a second: a path that the guard has judged is safe only for code that opens it the way the guard did; another program may see more in it, so what crosses that boundary must be bytes, not a name. The laws that held on first check (monotonicity, order independence, ancestor closure, narrowing, search implies listed, denied is absent, the round trips) are as informative as the failures, since they turn long-standing assumptions into checked statements.

10 Refinement and Summary

10.1 From the model to the code

Abstract	Concrete
fold	<code>rules.Fold (Go); fold in web/src/lib/filter.ts</code>
Match, MatchBelow	<code>Set.Match, Set.MatchBelow in internal/rules</code>
Deny, Hide	the two <code>rules.Set</code> values held by <code>guard.Guard</code>
canon, real	<code>canonPath, filepath.EvalSymlinks in internal/guard</code>
Ok, Open	<code>Guard.open</code> , with violation and <code>deniedByAncestor</code>
List, Visible	<code>Guard.ListFunc, Guard.entry</code>
Search	<code>Guard.Search (uses entry)</code>
sniff	<code>sniffImage, OpenPDF, ListArchive</code>
cmp, sort	<code>compareRows, sortRows in sort.ts</code>
filter	<code>matchesName in filter.ts</code>
toHash, fromHash	<code>pathToHash, parseHash in format.ts</code>
goto	<code>resolveGoto in goto.ts</code>
resolveLink	<code>resolveLink in markdown.ts</code>

Every HTTP-facing endpoint reaches the file system only through `Guard`; a test fails the build if an HTTP-facing package imports `os`, `io/fs`, `syscall` or `path/filepath`. This is what allows Laws 5.3 and 5.6 to be stated once for the guard rather than once per endpoint.

10.2 How to run the checks

```

go test -race ./...           # rule, guard and server laws
cd web && npm test             # frontend laws
node --test e2e/chrome.test.mjs # end-to-end (Safari: e2e/safari.test.mjs)
make -C specs/algebra         # build this document

```

10.3 Summary

- The deny rules form a set of exclusions: monotone, order independent, closed under descent, and invariant under every spelling that the file system equates.
- Every endpoint agrees with one access decision; listing implies openable, search implies listed, and denied paths are absent and look missing.
- Content-typed endpoints decide from bytes, disclose names only, and are bounded; Quick Look is given only bytes that `Open` would serve.
- The frontend order is a total preorder independent of input order, and the filter, hash and link functions are the exact counterparts of their server-side definitions.
- Eighteen defects were found and fixed; the laws that hold are recorded with the checks that establish them.

The document is intended to be read alongside `SECURITY.md` and the product functional and design specifications, which sit in the enclosing `specs` folder: those state what is promised and how it is built, and this states, precisely, the properties from which the promise follows.